

Relatório Técnico — Desafio Arcade: Esteira CI/CD do jogo Sinal Aberto

Faculdade SENAI · Integração e Entrega Contínua – DevOps · 2º ADS · Avaliação SAP1-DEVOPS

09/10/2026

Identificação

Integrante	RA	Papel
Joao Pedro Fonseca Alves de Carvalho	26179863	Plataforma e Release
João Gabriel Floriano Olimpio	26179859	Produto e Game Design
Kauan Alejandro da Rosa	26179867	Desenvolvimento e Qualidade
Renan Ramos Capeleti	26180895	SRE e Segurança

- **Repositório:** <https://github.com/Joaopedro0s/sinal-aberto>
- **Jogo em produção:** <https://joaopedro0s.github.io/sinal-aberto/>
- **Homologação:** <https://joaopedro0s.github.io/sinal-aberto/hml/>
- **Release v1.0.0:** <https://github.com/Joaopedro0s/sinal-aberto/releases/tag/v1.0.0>
- **Execução verde da esteira (commit → produção):**
<https://github.com/Joaopedro0s/sinal-aberto/actions/runs/38017114763>

1. O produto: Sinal Aberto - O Controlador de Trânsito

Jogo educativo para celular em que o jogador programa, com blocos de código, os semáforos de um cruzamento. A cada fase ele aprende um conceito de programação (sequência, if, if/else com ou, variáveis e, por fim, leitura e correção de JavaScript real) e vê o resultado da própria lógica: o trânsito flui ou os carros colidem. O público é o de jovens iniciantes do concurso Framework Arcade.

Stack: **Phaser 3 + TypeScript + Vite**, site estático com caminhos relativos (roda em /hml/ e em /releases/<sha>/), build em arquivo único (abre offline pelo index.html do build.zip, 389 KB). A lógica do jogo fica em src/core/ (interpretador de AST sem eval, simulação determinística com semente, objetivos e estrelas, gerador de JavaScript), separada da renderização em src/scenes/ — o que permite testar as regras sem navegador. Arte e sons são gerados por código (sem ativos de terceiros) e o jogo não coleta dado pessoal (progresso só no localStorage).

2. Arquitetura da esteira e decisões

Toda a esteira roda no GitHub, sem custo: GitHub Actions (ubuntu-latest) e GitHub Pages (branch gh-pages).

```
feature/* → Pull Request (revisão de outro integrante + CI) → main
├─ ci: npm ci → lint → testes (JUnit + cobertura) → gitleaks → npm audit
│   → SBOM (CycloneDX) → build → GDD.pdf (Pandoc) → build.zip + SHA-256
├─ deploy-hml: publica em /hml/ → espera o Pages servir o SHA novo → E2E Playwright
├─ deploy-prd (environment "producao", revisor obrigatório):
│   publica em /releases/<sha>/ → canário 10% (rollout.json) → smoke
│   → falhou? rollback automático | passou? promove a 100%
└─ release (tag vX.Y.Z): GitHub Release com build.zip, checksum, SBOM e GDD.pdf
```

Workflows de apoio: qualidade.yml (reprodutibilidade e limite de 25 MB),
monitor.yml (sondas a cada 15 min), rollback.yml (manual), notificar-falha.yml

Um artefato, vários ambientes. O build.zip é gerado uma única vez no job ci, enviado como artefato build-<sha> e baixado pelos jobs de homologação e produção; nada é recompilado entre ambientes.

Entrega progressiva por ponteiro. Cada versão fica numa pasta imutável releases/<sha>/. O index.html da raiz é um carregador que lê rollout.json (estavel, anterior, canario, percentual), sorteia o canário pelo percentual e guarda a escolha no navegador, para o jogador não trocar de versão no meio da sessão. Rollback é trocar o ponteiro estavel para anterior (scripts/rollout.sh rollback), sem recompilar e sem tirar a URL do ar. Todo commit no gh-pages é feito pela pipeline, com autor e motivo na mensagem.

Alternativas avaliadas e descartadas

Decisão	Escolha	Descartado	Motivo
Engine	Phaser 3 + TypeScript	Godot 4, Unity	Godot/Unity geram builds web de dezenas de MB, dificultam cobertura com Vitest e E2E com Playwright; Unity exige licença no CI
Hospedagem	GitHub Pages	Vercel, Netlify	Deploy automático fora da pipeline violaria a regra “produção só pela esteira”; Pages permite pastas por release e ponteiro
Estratégia	Canário 10%	Azul-verde	Mesmo mecanismo de pastas; o canário permite expor a versão nova a poucos jogadores antes de promover
Execução do código do jogador	Interpretador de AST	eval()	Segurança e testabilidade (cada passo é uma função pura)
Geração de jogos por IA hospedada	Código próprio no repositório	Rosebud AI, Combos	Não permitem baixar o código no plano gratuito e publicariam fora da esteira

3. Evidências por integrável

ID	Integrável	Evidência	Situação
INT-01	Repositório e fluxo Git	SQUAD.md, PRs #11-#22 revisados, Conventional Commits, tags v1.0.0	Entregue
INT-02	GDD como código	docs/gdd.md (com telas reais em docs/img/) → GDD.pdf gerado na pipeline e anexado à release	Entregue
INT-03	Pipeline CI/CD	.github/workflows/esteira.yml; run 60 verde do commit à produção	Entregue
INT-04	Testes automatizados	88 de unidade, 29 de integração, 8 E2E; cobertura de src/core 99,8% (linhas); relatório em docs/qualidade.md	Entregue
INT-05	Segurança, licenças e IA	gitleaks, npm audit, SBOM, Dependabot, THIRD_PARTY.md, AI-USAGE.md (tabela), SECURITY.md	Entregue
INT-06	Release versionada	build.zip + SHA-256 + version.json; GitHub Release v1.0.0; qualidade.yml verifica reprodutibilidade e tamanho	Entregue
INT-07	Ambientes e rollback	gh-pages com /hml/, /releases/<sha>/, rollout.json; rollback automático no smoke e rollback.yml manual; testes do carregador	Entregue (rollback cronometrado na banca)
INT-08	Monitoramento e DORA	monitor.yml com sondas reais a cada 15 min, histórico no branch observabilidade, Issues JogoForaDoAr/LatenciaAlta com fechamento automático; métricas DORA calculadas manualmente (seção 4)	Parcial: scripts/dora.mjs e painel /status/ ainda com dados fixos
INT-09	Build pública e pacote	URL pública, build.zip, GDD.pdf, SBOM e checksum na release	Parcial: scripts/triagem.sh ainda simplificado
INT-10	Pitch e relatório	Roteiro e legendas em docs/pitch/; este relatório	Parcial: vídeo pitch.mp4 pendente

4. Métricas DORA do período (05/10 a 09/10/2026)

Os valores abaixo foram levantados manualmente a partir do histórico do GitHub Actions e do gh-pages, porque o scripts/dora.mjs ainda não lê a API.

Métrica	Valor	Como foi medido
Frequência de deploy em produção	2 deploys em 5 dias (05/10 e 09/10)	execuções bem-sucedidas do job deploy-prd
Lead time de mudança	≈ 10 min (merge do PR #22 → produção, incluindo aprovação manual)	horário do merge vs. fim do deploy-prd do run 60
Taxa de falha de mudança	1 de 3 mudanças relevantes no main quebrou a esteira (≈ 33%)	merge do PR #10 (TypeScript 7) e do GDD com símbolos Unicode
Tempo de recuperação	≈ 4 dias (05/10 → 09/10)	da quebra do npm ci até a primeira execução verde completa

Comparação com a linha de base da Carparts (lead time de 11 dias): o lead time caiu de dias para minutos, porque a mesma pipeline vai do merge à produção sem passos manuais além da aprovação do environment.

O que as métricas mostraram e a decisão tomada. A produção nunca saiu do ar — a falha ficou contida na esteira, que se recusou a publicar. Mas o main ficou 4 dias sem conseguir publicar porque (1) um PR do Dependabot com atualização *major* (TypeScript 5 → 7) foi aceito e (2) a proteção da main não exigia o status check ci, então PRs com o CI vermelho entraram. Decisões: exigir o check ci na proteção da main, não aceitar atualizações *major* do Dependabot sem revisão (política em SECURITY.md) e corrigir o passo do PDF (--resource-path=docs e remoção de símbolos que o pdflatex não suporta, PR #22). Depois disso, o commit chegou à produção em ≈ 10 minutos.

5. Retrospectiva

O que funcionou

- Separar regras (src/core) de renderização deu cobertura alta e testes rápidos; os E2E rodam contra a homologação real.
- O modelo de pastas imutáveis + rollout.json fez o rollback virar uma troca de ponteiro, testável e auditável no histórico do gh-pages.
- Os gates da pipeline cumpriram o papel: o npm ci barrou a dependência incompatível e o passo do GDD barrou um PDF que não seria gerado — nada quebrado chegou ao jogador.

O que não funcionou

- Concentração de commits em um integrante no início; a divisão por papéis e PRs só aconteceu na reta final.
- Aceitar um PR do Dependabot sem olhar o CI e sem exigir o check na main.
- Os scripts dora.mjs e triagem.sh ficaram simplificados e o vídeo do pitch não ficou pronto a tempo.

O que mudaríamos

- Configurar a proteção da main (PR + 1 revisão + check ci obrigatório) no primeiro dia.
- Dividir as tarefas por integrável desde a formação do squad.
- Automatizar o DORA pela API do GitHub e rodar a triagem agendada até a banca.

6. Contribuição de cada integrante

Integrante	Contribuições (PRs)
Joao Pedro Fonseca Alves de Carvalho	Esqueleto e esteira (commits iniciais), troca do jogo para o Sinal Aberto (#11), nome do projeto (#12), correção do GDD.pdf na esteira (#22), tag e release v1.0.0, aprovações de deploy
João Gabriel Floriano Olimpio	Telas reais do jogo no GDD (#19), AI-USAGE.md em tabela (#20), roteiro e legendas do pitch (#21)
Kauan Alejandro da Rosa	Testes de casos de borda e docs/qualidade.md (#13), workflow de qualidade com reprodutibilidade e limite de tamanho (#14), testes do carregador de versões (#15)
Renan Ramos Capeleti	SECURITY.md (#16), monitoramento real com sondas e alertas (#17), notificação de falha da pipeline (#18)

Links: <https://github.com/Joaopedro0s/sinal-aberto/pulls?q=is%3Apr>

7. Uso de IA

Foram usadas ferramentas de IA generativa (Gemini 3.1 Pro no Antigravity, Claude e Claude Code) para ideação, geração de código, testes, documentação e diagnóstico da esteira. Todo uso está registrado em AI-USAGE.md, e o código gerado passou pelos mesmos testes, varreduras e revisões de PR que o código escrito à mão.